C# бағдарламалау негіздері

EduLibrary Demo Content

Айнымалылар, шарттар, циклдер, әдістер және объектілер арқылы шағын, тексерілетін C# бағдарламасын құруға арналған түпнұсқа қысқаша сабақтар.

Қысқаша сабақтар · 2026 · Қазақша

Оқу жолы

Бұл материалды қалай оқу керек

Әр сабақтағы ұғымды оқып, мысалды қолмен жүргізіңіз. Содан кейін тәжірибе тапсырмасын өзіңіз орындап, соңғы беттегі түсіндірмемен салыстырыңыз. Код үзінділері оқу үшін ықшамдалған; олар толық өндірістік жүйені алмастырмайды.

1. Бағдарлама және дерек түрлері

2. Шарт арқылы шешім қабылдау

3. Цикл және жинақ

4. Әдіс және жауапкершілік

5. Объект және күйді қорғау

Шағын жоба және жауаптарды тексеру

Авторлық құқық және шығу тегі

Автор: EduLibrary Demo Content. 2026 жылы EduLibrary демонстрациясы үшін жаңадан жазылған түпнұсқа қазақша сабақтар. Коммерциялық кітаптан көшірілген мәтін, тарау не файл қолданылған жоқ. Бұл материал бес дербес шағын оқу құралының бірі.

Лицензия: Creative Commons Attribution 4.0 International (CC BY 4.0). Авторды және осы лицензияны көрсетіп пайдалануға болады. Лицензия: https://creativecommons.org/licenses/by/4.0/ . Бастапқы мәтін мен генератор осы жобаның Data/DigitalDemo/content.json және tools/generate_digital_demo.py файлдарында сақталған.

1. Бағдарлама және дерек түрлері

Бағдарлама кіріс дерегін қабылдап, анық ереже бойынша нәтиже береді. Мысалы, оқу залының бос орындарын есептейтін бағдарлама жалпы орын санынан бос емес орын санын шегереді. Есептеудің алдында деректің мағынасы мен түрін белгілеу қажет. Бүтін сан үшін int, бөлшек ақша үшін decimal, мәтін үшін string, шарт үшін bool қолданылады.

Айнымалының атауы оның қызметін түсіндірсін: availableSeats атауы x атауына қарағанда түсінікті. Нақты әлемдегі ережені кодпен көрсетіңіз: орын саны теріс болмауы керек. C# түрлерді тексергенімен, барлық бизнес ережені өздігінен білмейді. Енгізілген мәтінді санға түрлендіргенде TryParse нәтижесін тексеру - қалыпты қателерді басқарудың ыңғайлы тәсілі.

Мысалды талдау

```
int totalSeats = 40;
int occupiedSeats = 27;
int availableSeats = totalSeats - occupiedSeats;
Console.WriteLine(availableSeats);
```

Өзіңіз орындаңыз

Залда 48 орын бар, оның 19-ы бос емес. Бос орын санын есептеңіз. Одан кейін occupiedSeats мәні 50 болса, бағдарлама қандай тексеру жасауы керек екенін жазыңыз.

2. Шарт арқылы шешім қабылдау

Шарт - true не false болатын өрнек. if тармағы белгілі бір ереже орындалғанда әрекет жасайды. Кітапты беру үшін оқырман белсенді және дана қолжетімді болуы керек делік. Осы екі шартты && арқылы байланыстырамыз. || кемінде бір шарт жеткілікті болғанда қолданылады. Күрделі өрнекте жақша қолдану оқуды жеңілдетеді.

Шекаралық мәндерге назар аударыңыз. Жас шегі 18 болса, age > 18 өрнегі 18 жастағы адамды қабылдамайды; age >= 18 қабылдайды. Шартты тек сәтті мысалмен емес, шекараның екі жағындағы деректермен де тексеріңіз. Тармақтар тым көбейсе, алдымен қате жағдайды анықтап, ерте қайту тәсілі негізгі әрекетті анық көрсетуге көмектеседі.

Мысалды талдау

```
bool activeStudent = true;
bool copyAvailable = false;
if (activeStudent && copyAvailable)
   Console.WriteLine("Ready");
else
   Console.WriteLine("Check eligibility");
```

Өзіңіз орындаңыз

Температура 0-ден төмен болса мұз қаупін, 0 мен 30 аралығында қалыпты жағдайды, 30-дан жоғары болса ыстықты көрсететін үш тармақ құрыңыз. Дәл 0 және 30 мәндерін тексеріңіз.

3. Цикл және жинақ

Бір әрекетті бірнеше дерекке орындау үшін цикл қажет. foreach жинақтағы әр элементті кезекпен қарайды және индексті қолмен басқаруды талап етпейді. for индекс қажет болғанда ыңғайлы. while тоқтау шарты алдын ала белгісіз қайталануға жарайды, бірақ шарт бір кезде false болуы тиіс.

Жинақтағы мәндердің қосындысын табу үшін аккумулятор қолданылады. Оны цикл алдында нөлден бастап, әр қадамда жаңартамыз. Бос жинақта цикл бірде-бір рет орындалмайды, сондықтан бастапқы мән нәтижеге әсер етеді. Орташа мәнді есептегенде элемент саны нөл емес екенін тексеріңіз. Цикл ішінде жинақ құрылымын өзгерту қате туғызуы мүмкін; өзгерістерді бөлек жоспарлаған дұрыс.

Мысалды талдау

```
int[] pages = [12, 18, 10];
int total = 0;
foreach (int count in pages)
    total += count;
Console.WriteLine(total);
```

Өзіңіз орындаңыз

Күн сайын оқылған беттер: 8, 14, 0, 21. Қосындыны және нөлден үлкен күндер санын есептеңіз. Бос массив үшін нәтиже қандай болатынын түсіндіріңіз.

4. Әдіс және жауапкершілік

Әдіс бір мағыналы әрекетті атқарады, параметр қабылдап, нәтиже қайтарады. Ол қайталанатын логиканы бір жерге жинайды. Мысалы, кешіккен күн санын есептеу экранға мәтін шығарудан бөлек болғаны дұрыс. Сонда есептеу әдісін веб бетте де, консольде де қайта қолдануға болады.

Параметрлер мен қайтарылатын мән әдістің келісімі болып саналады. Жасырын сыртқы күйді өзгерту бұл келісімді түсінуді қиындатады. Әдіс атауы етістіктен басталса, оның әрекеті анық көрінеді. Бір әдіс деректі тексеріп, файл жазып, хат жіберіп, баға есептесе, оны бірнеше шағын жауапкершілікке бөлуді қарастырыңыз.

Мысалды талдау

```
static int Remaining(int total, int used)
{
    if (total < 0 || used < 0 || used > total)
        throw new ArgumentOutOfRangeException();
    return total - used;
}
```

Өзіңіз орындаңыз

Екі бүтін санның үлкенін қайтаратын MaxOfTwo әдісін жазыңыз. Тең екі сан үшін не қайтаратынын белгілеңіз. Үш шағын тексеру мысалын беріңіз.

5. Объект және күйді қорғау

Класс бір ұғымның күйі мен әрекетін біріктіреді. Кітапхана картасының иесі, жарамдылық мерзімі және белсенділігі бар. Объект осы нақты картаның күйін сақтайды. Бір класс бойынша жасалған екі объектінің мәндері бір-бірінен тәуелсіз болады. Бұл ұғымды жай сөздік анықтама ретінде емес, домен моделін құру тәсілі ретінде қарастырыңыз.

Қасиетті барлық жерден өзгертуге рұқсат беру ережелерді айналып өтуге мүмкіндік береді. Мысалы, қордағы дана санын азайту тек саны нөлден үлкен кезде орындалсын. Осы әрекетті әдіске жинап, setter қолжетімділігін шектеуге болады. Мұрагерлікке асықпаңыз: қарапайым құрамдастыру көбіне жеткілікті. Алдымен нақты жауапкершілігі бар шағын модель құрыңыз.

Мысалды талдау

```
class ReadingGoal
{
   public int PagesRead { get; private set; }
   public void Add(int pages)
   {
      if (pages <= 0) throw new ArgumentException();
      PagesRead += pages;
   }
}
```

Өзіңіз орындаңыз

ReadingGoal объектісіне 12 және 8 бет қосыңыз. Нәтижені болжаңыз. Нөл және теріс мәнге рұқсат берілсе, есептің мағынасы қалай бұзылатынын түсіндіріңіз.

Жоба және өзіндік тексеру

Шағын оқу күнделігін құрыңыз: күн сайынғы бет санын жинаққа қосу, жалпы оқылғанын есептеу және мақсатқа қалған бет санын көрсету. Теріс енгізуді қабылдамаңыз. Есептеу әдістерін Console операцияларынан бөліңіз. Бос күнделік, дәл орындалған мақсат және мақсаттан асу жағдайларын тексеріңіз.

Тапсырмаларға түсіндірме

1. Бос орын саны 29. occupiedSeats 0 мен totalSeats аралығында болмаса, есептеуді тоқтатып, енгізу қатесін көрсету керек.

2. Бірінші шарт temperature < 0, екінші шарт temperature <= 30 болуы мүмкін. Екінші шартқа келгенде теріс мәндер бұған дейін алынып тасталған. 0 және 30 қалыпты тармаққа түседі.

3. Барлығы 43 бет, оқу болған күн саны 3. Қосынды мен санауыш нөлден басталса, бос массив екеуіне де 0 береді.

4. a >= b болса a, әйтпесе b қайтарылады. Тең сандардың кез келгені дұрыс. Мысалдар: (2,5)->5, (7,1)->7, (4,4)->4.

5. PagesRead 20 болады. Теріс мән оқу тарихын азайтып, тек қосуға арналған әрекеттің ережесін бұзады. Қате дерек әдістің шекарасында қабылданбауы керек.

Келесі қадам: жобаңызды басқа адамға түсіндіріп көріңіз. Қандай кіріс қабылданады, қандай ереже қорғалады және нәтижені қалай тексересіз?